

Navigation2 Single-Point Navigation

Navigation2 Single-Point Navigation

1. Course Content
2. Introduction to Navigation2
3. Case Demo
 - 4.2 Viewing the Costmap
5. Navigation Explanation
 - 5.1 Core Steps of the Navigation Flow
 - 5.1.1 Phase 1: System Initialization and Map Loading
 - 5.1.2 Phase 2: Sensor Data Processing and Environment Perception
 - 5.1.3 Phase 3: Path Planning (Global Planning and Local Planning)
 - 5.1.4 Phase 4: Control Execution and Behavior Decision
 - 5.2 Key Technical Principles
 - 5.2.1 Costmap
 - 5.2.2 AMCL Localization Algorithm
 - 5.3 Path Planning
 - 5.3.1 Global Path Planning
 - 5.3.2 Local Path Planning
6. Navigation Parameter Configuration
 - 6.1 Navigation Speed Tuning
 - 6.2 Obstacle Avoidance Tuning
7. Source Code Analysis

[!IMPORTANT]

Note: This course requires at least one grid map for navigation. If you have not built a map yet, please refer to the previous SLAM mapping course to build a grid map first.

1. Course Content

Basics:

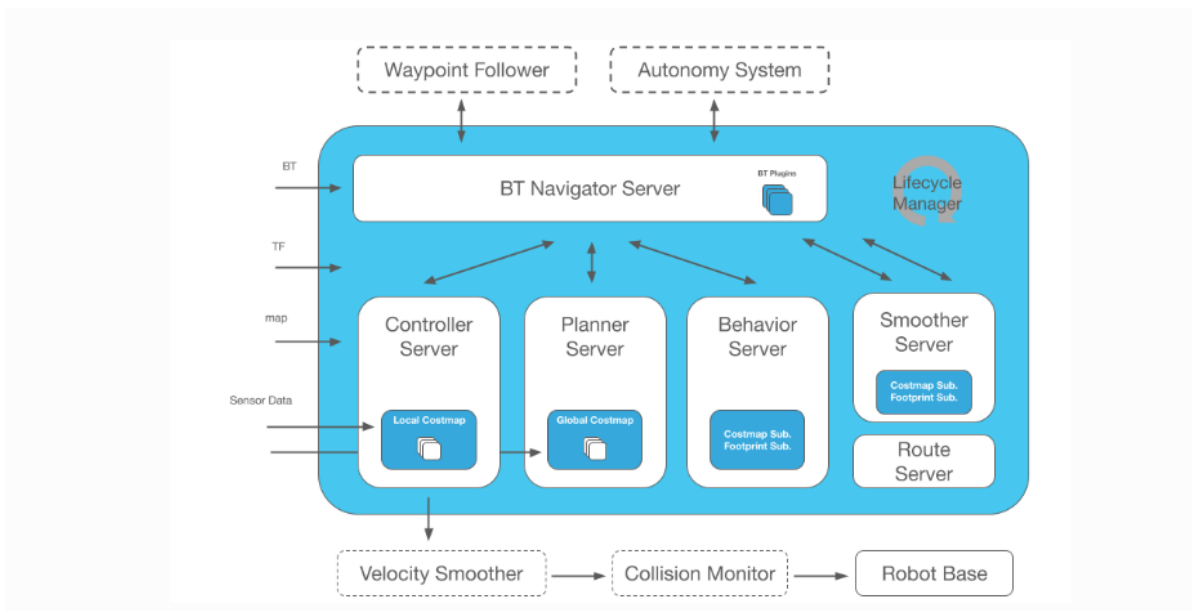
- Run the example program, and rviz will load the map. In the rviz interface, use the "2D Goal Pose" tool to assign a goal point to the car. Based on its surroundings, the car plans a path and follows it to the destination. If it encounters obstacles on the way, it will avoid them autonomously and stop after reaching the destination.

Advanced:

- Understand the concepts of the global costmap and local costmap, tuning of navigation speed and obstacle avoidance, and the principle of the AMCL localization algorithm.

2. Introduction to Navigation2

- Navigation2 overall architecture diagram



Navigation2 provides the following tools:

- A tool for loading, providing, and storing maps (Map Server)
- A tool for localizing the robot on the map (AMCL)
- A path planning tool that moves the robot from point A to point B while avoiding obstacles (Nav2 Planner)
- A tool for controlling the robot while following a path (Nav2 Controller)
- A tool for converting sensor data into a costmap representation of the robot's world (Nav2 Costmap 2D)
- A tool for building complex robot behaviors using behavior trees (Nav2 Behavior Trees and BT Navigator)
- A tool for computing recovery behaviors when failures occur (Nav2 Recoveries)
- A tool for following sequential waypoints (Nav2 Waypoint Follower)
- A tool for managing server lifecycle and watchdogs (Nav2 Lifecycle Manager)
- Plugins that enable user-defined algorithms and behaviors (Nav2 Core)

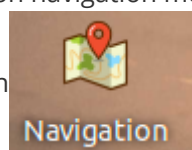
Navigation 2 (Nav 2) is the navigation framework that ships with ROS 2. Its purpose is to enable a mobile robot to move from point A to point B in a safe way. Therefore, Nav 2 can perform behaviors such as dynamic path planning, computing motor velocities, avoiding obstacles, and recovery.

Nav 2 uses behavior trees (BT) to call modular servers to complete an action. An action can be computing a path, control efforts, recovery, or other navigation-related actions. These actions are performed by independent nodes that communicate with the behavior tree (BT) through action servers.

3. Case Demo

- The default AMCL localization navigation mode can be quickly launched by double-clicking

the desktop application icon



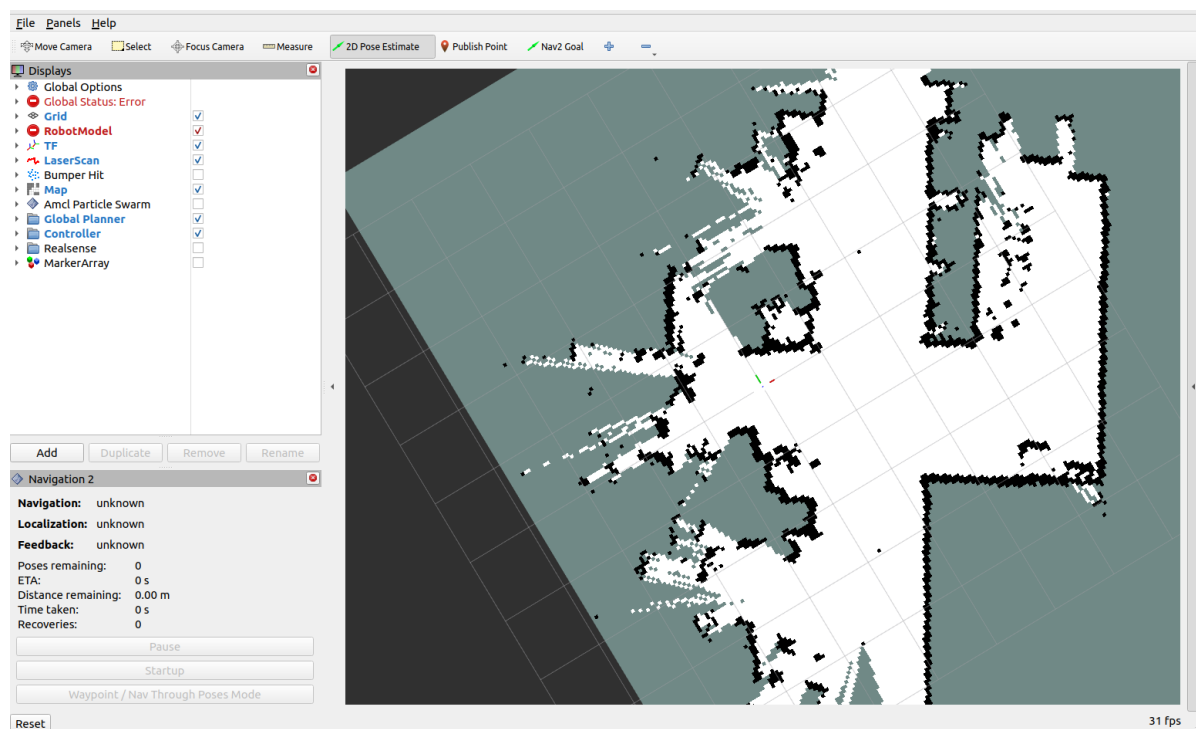
- Command-line launch method

```
ros2 launch microros_v2_bringup nav2.launch.py
```

[!NOTE]

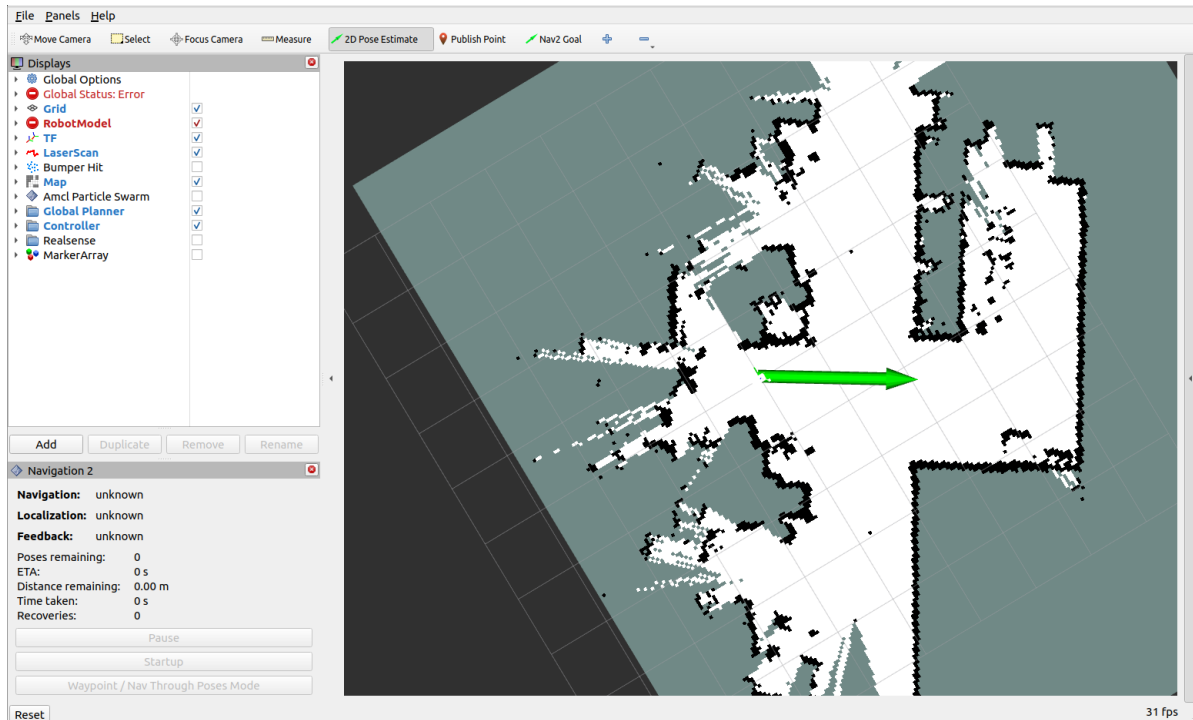
Optional launch argument descriptions:

- map
 - Map name, default value `map.yaml`. The map must be saved in the system `~/map` directory.
- gui
 - Whether to launch the rviz display window, default value `True`.
- pose_map
 - Posegraph pose map name. If `slam_toolbox` is selected as the localization mode for navigation, the pose map name must be provided. Default value `map`. Only the name is needed, without a suffix; the corresponding pose map will be automatically searched in the system `~/map` directory.
- state_filename
 - pbstream pose map name. If `cartographer` is selected as the localization mode for navigation, the pose map name must be provided. Default value `map.pbstream`.
- localization_mode
 - Localization mode, default is `amcl`. Options also include `cartographer` localization, `slam_toolbox` localization, and `rtabmap` localization.
- log_level
 - Log level, default is `info`.
- namespace
 - Namespace, default is an empty string.

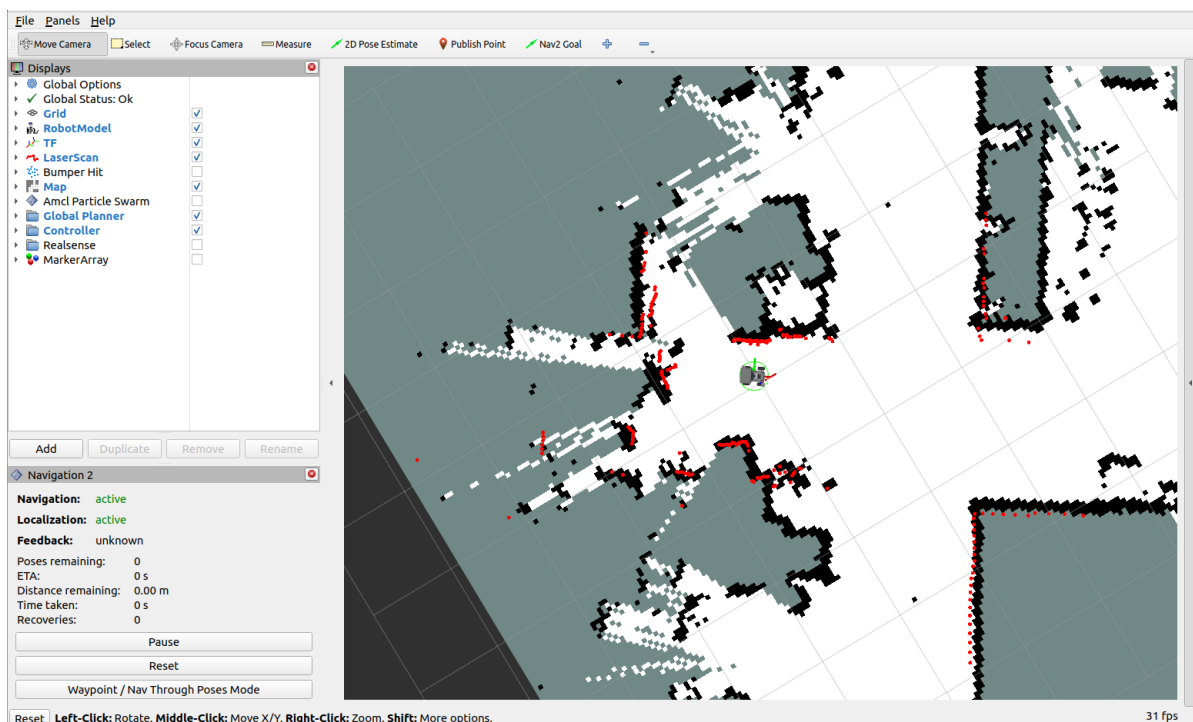


- At this point you can see that the map has been loaded. Then click "2D Pose Estimate" to set the initial pose of the car. Based on the car's actual position in the real environment, click

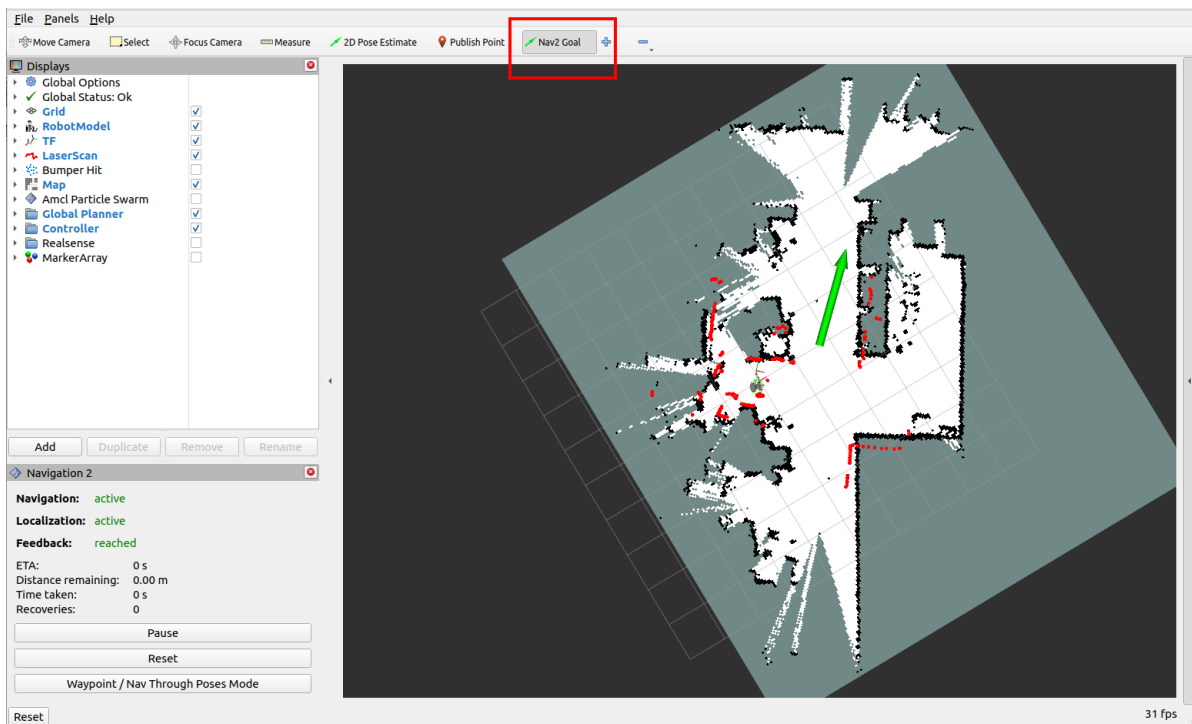
and drag with the mouse in rviz; the car model then moves to the position you set. As shown in the figure below, if the area scanned by the lidar roughly coincides with the actual obstacles, the pose is accurate.



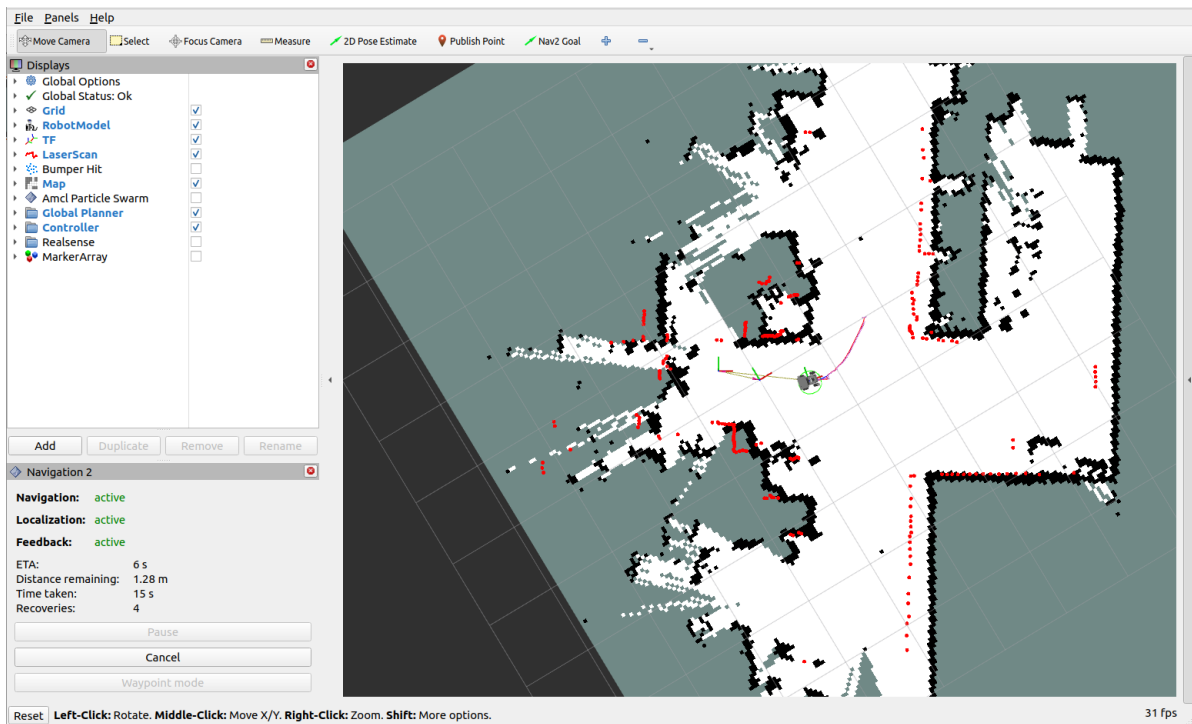
- After pose initialization is complete, the robot model and the red 2D lidar point cloud will appear in the rviz interface.



- For single-point navigation, click the "2D Goal Pose" tool, then use the mouse in rviz to select a goal point and goal orientation, and then release.



- Based on its surroundings, the car plans a path and follows it to the goal point.

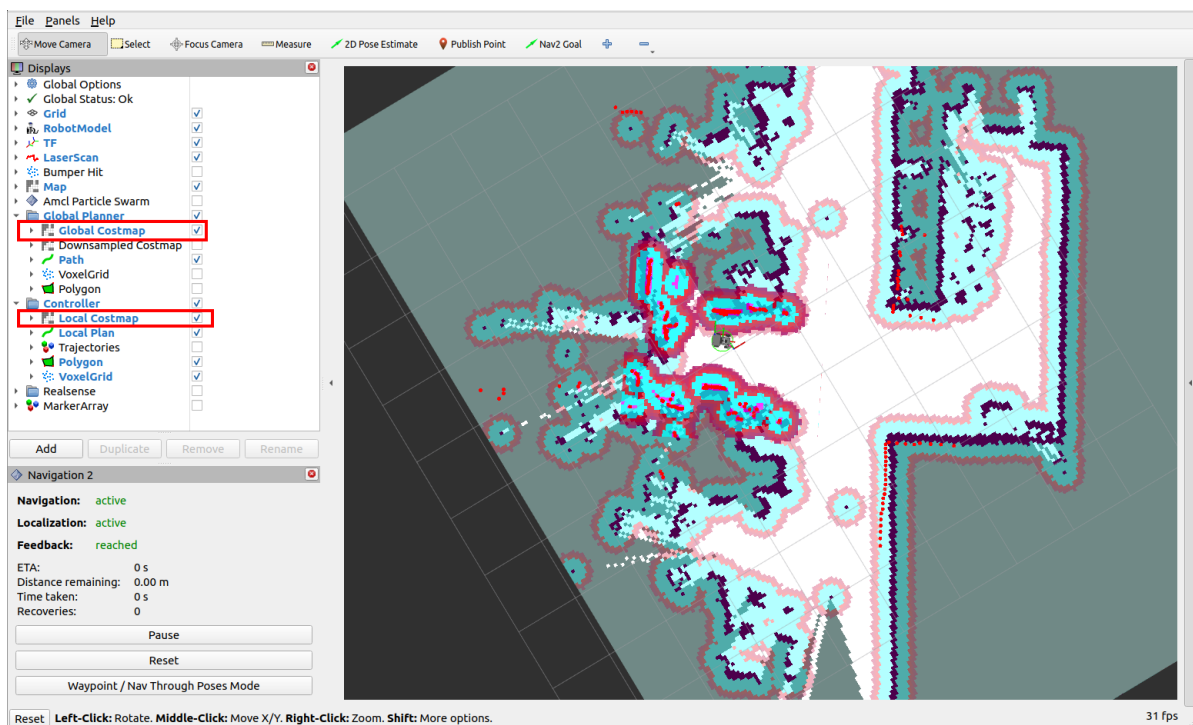


- After the robot successfully reaches the goal point, the car terminal will prompt "Goal succeeded", indicating that navigation was successful.

```
jetson@yahboom: ~  
request to clear except a region the global_costmap  
[component_container_isolated-1] [INFO] [1749801916.491747885] [local_costmap.local_costmap]: Received r  
equest to clear except a region the local_costmap  
[component_container_isolated-1] [WARN] [1749801916.856635255] [planner_server]: Planner loop missed its  
desired rate of 5.0000 Hz. Current loop rate is 3.3122 Hz  
[component_container_isolated-1] [INFO] [1749801916.945246404] [controller_server]: Passing new path to  
controller.  
[component_container_isolated-1] [INFO] [1749801917.945237102] [controller_server]: Passing new path to  
controller.  
[component_container_isolated-1] [INFO] [1749801918.945250444] [controller_server]: Passing new path to  
controller.  
[component_container_isolated-1] [INFO] [1749801919.492802205] [global_costmap.global_costmap]: Received  
request to clear except a region the global_costmap  
[component_container_isolated-1] [INFO] [1749801919.494641919] [local_costmap.local_costmap]: Received r  
equest to clear except a region the local_costmap  
[component_container_isolated-1] [INFO] [1749801920.145240124] [controller_server]: Passing new path to  
controller.  
[component_container_isolated-1] [INFO] [1749801921.145251451] [controller_server]: Passing new path to  
controller.  
[component_container_isolated-1] [INFO] [1749801921.560350643] [controller_server]: Reached the goal!  
[component_container_isolated-1] [INFO] [1749801921.594495227] [bt_navigator]: Goal succeeded  
[component_container_isolated-1] [INFO] [1749801922.495633489] [global_costmap.global_costmap]: Received  
request to clear except a region the global_costmap  
[component_container_isolated-1] [INFO] [1749801922.497469011] [local_costmap.local_costmap]: Received r  
equest to clear except a region the local_costmap
```

4.2 Viewing the Costmap

- To view the global costmap and local costmap, find Global Costmap under the Global Planner group in the left configuration panel, check it, and the global costmap will be displayed.
- Similarly, find the Local Costmap option under the Controller group, check it, and the local costmap will be displayed.



5. Navigation Explanation

5.1 Core Steps of the Navigation Flow

5.1.1 Phase 1: System Initialization and Map Loading

Map Acquisition

- Load the pre-built grid map from `map_server`
- The map contains static obstacle information and serves as the base environment model for navigation.

Costmap Initialization

- **Global Costmap:** Based on the static map, used for global path planning.
- **Local Costmap:** Fuses the lidar `scan` topic in real time for dynamic obstacle avoidance.

5.1.2 Phase 2: Sensor Data Processing and Environment Perception

Sensor Data Input

- Lidar (`/scan`), IMU (`/imu/data`), and odometry (`/odom`) data are input through ROS topics.

Costmap Update

- The local costmap updates dynamic obstacle information in real time. Each grid cell computes its occupancy probability and cost based on sensor data (e.g., the closer to an obstacle, the higher the cost).
- Inflation: Expands the edges of obstacles to prevent the robot from getting too close to them.

5.1.3 Phase 3: Path Planning (Global Planning and Local Planning)

Global Path Planning

- Inputs: start point (the robot's current pose), goal point (the target pose), and the global costmap.
- The Dijkstra planner algorithm generates the optimal path from the start point to the goal point (a series of discrete coordinate points).

Local Path Planning and Tracking

The local planner `DWBLocalPlanner` generates short-term control commands executable by the robot, based on the global path and the local costmap.

5.1.4 Phase 4: Control Execution and Behavior Decision

Controller Output

- The controller converts the local path into the robot's linear velocity (`linear.x`) and angular velocity (`angular.z`), published through the `/cmd_vel` topic.
- Velocity smoothing: avoids sudden changes in robot motion and improves stability.

Behavior Tree Decision Flow

The behavior tree defines the priority and state-transition logic of the navigation task. A typical flow:

1. **Check whether the goal is reachable:** If global planning fails, a "recovery behavior" is triggered (e.g., replanning).
2. **Perform local obstacle avoidance:** When the local costmap detects an obstacle, temporarily deviate from the global path.

3. **Reach the goal:** When the error between the robot's pose and the goal pose is smaller than the threshold, the task is complete.

Recovery Behavior Mechanism

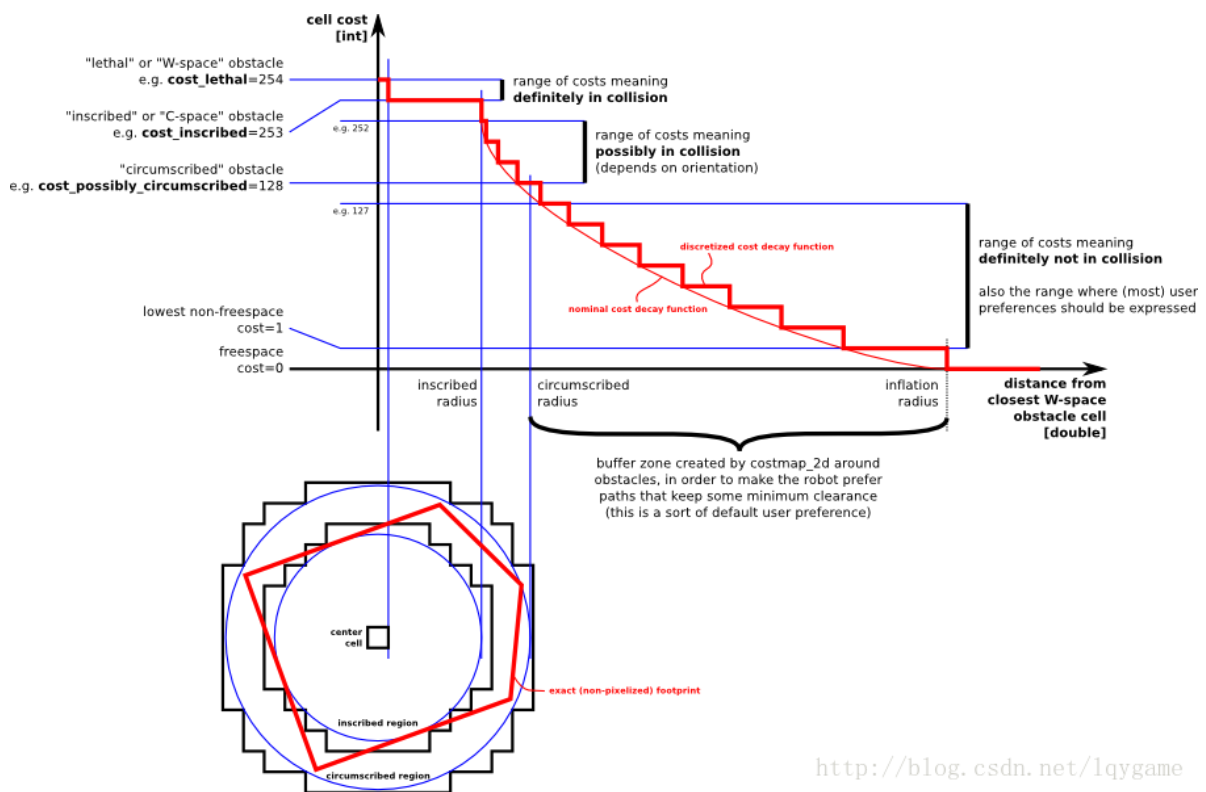
When navigation encounters an anomaly (e.g., the path is completely blocked), a preset recovery strategy is triggered (e.g., spinning to search for a new path, backing up and replanning).

5.2 Key Technical Principles

5.2.1 Costmap

- **Probabilistic grid representation:** Each grid cell stores an occupancy probability (0-1), updated with sensor data (e.g., lidar ray casting).
- **Multi-layer cost stacking:** static cost (map obstacles) + dynamic cost (obstacles detected by real-time sensors) + inflation cost (safety distance).

A costmap is a 2D or 3D map built and updated by the robot from collected sensor information. You can get a brief understanding from the figure below.



In the figure above, the red parts represent obstacles in the costmap, and the blue parts represent obstacles inflated by the radius of the robot's inscribed circle. The red polygon is the footprint (the vertical projection of the robot's outline). To avoid collisions, the footprint should not intersect the red parts, and the robot's center should not intersect the blue parts.

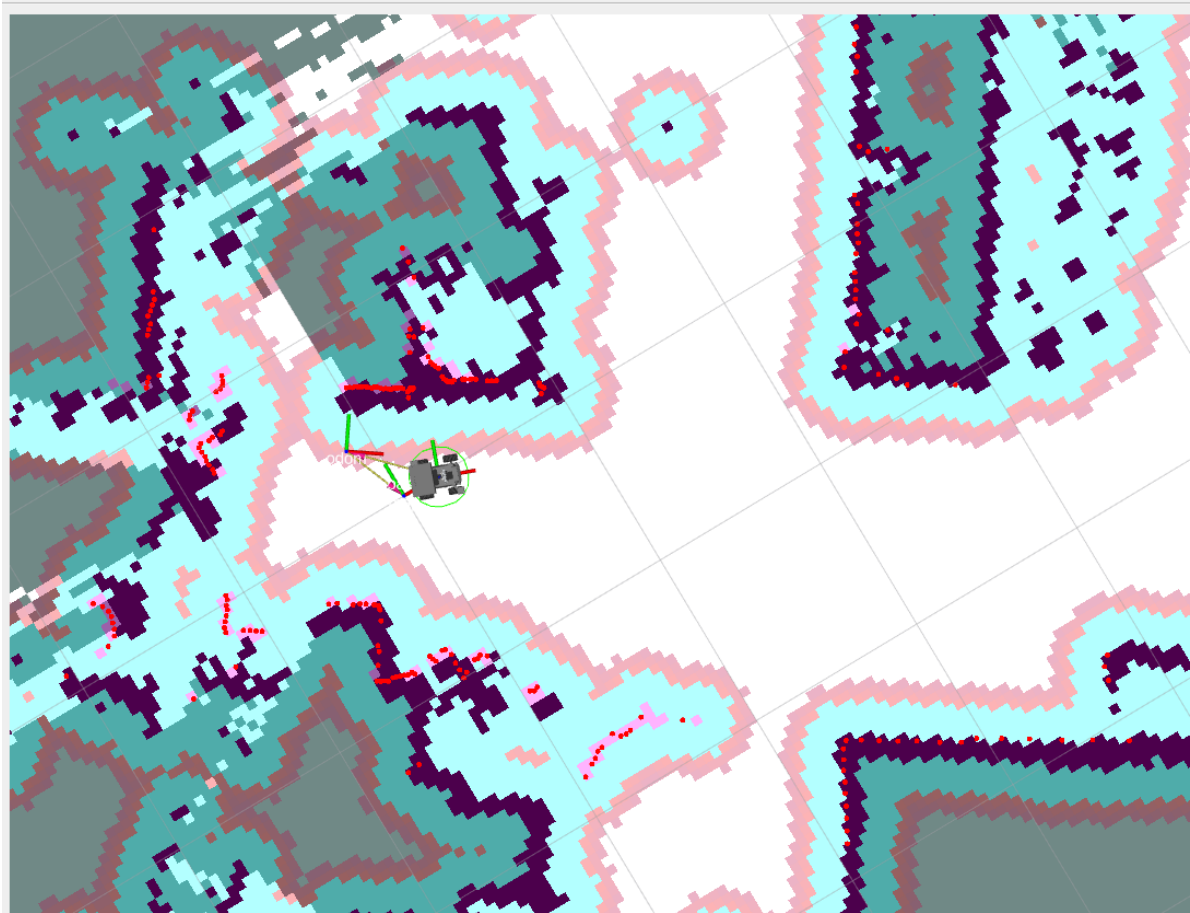
ROS's costmap uses a grid form, where each grid cell's value (cell cost) ranges from 0 to 255. There are three states: occupied (with obstacles), free space (no obstacles), and unknown area. The specific correspondence between states and values is shown in the figure below:

The figure above can be divided into five parts, where the red polygonal area is the robot's outline:

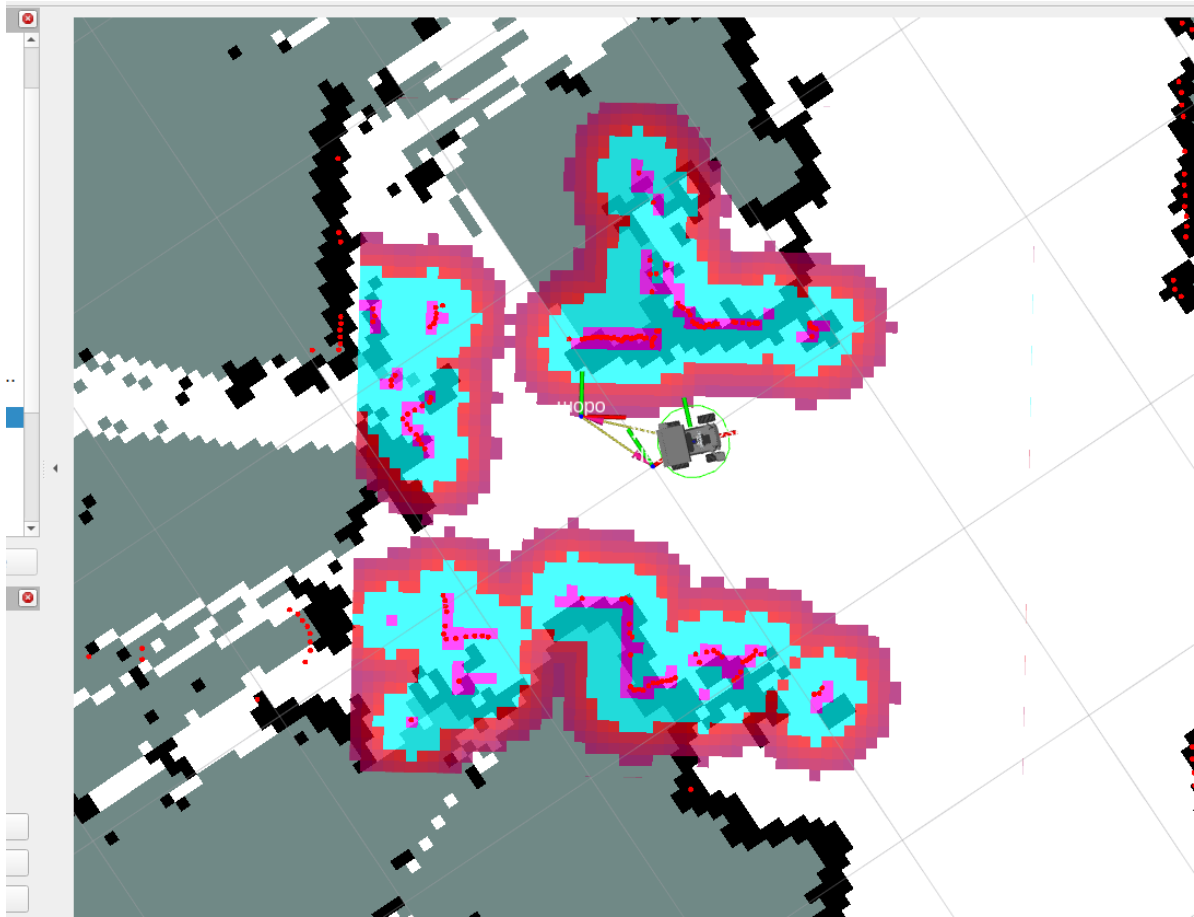
- **Lethal:** The robot's center coincides with the center of this grid cell, so the robot is guaranteed to collide with the obstacle.
- **Inscribed:** The grid cell's circumscribed circle is inscribed in the robot's outline, so the robot is also guaranteed to collide with the obstacle.

- Possibly circumscribed: The grid cell's circumscribed circle is externally tangent to the robot's outline, so the robot is essentially leaning against the obstacle; a collision is not certain.
- Freespace: Space without obstacles.
- Unknown: Unknown space.

Global costmap:



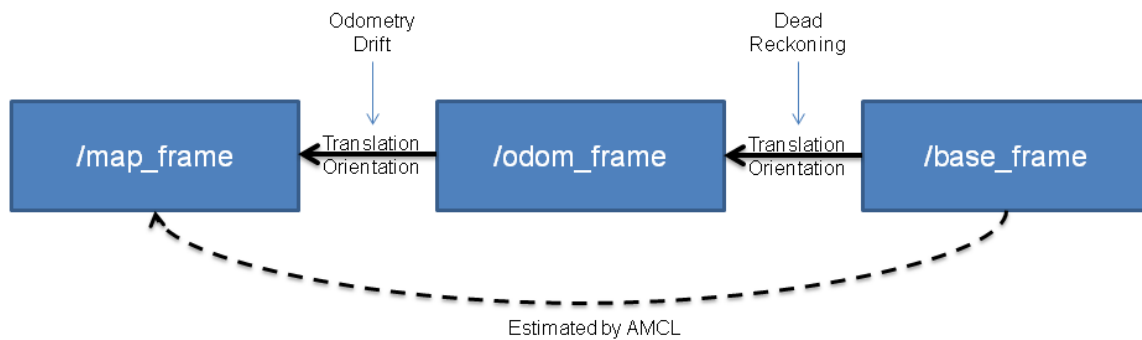
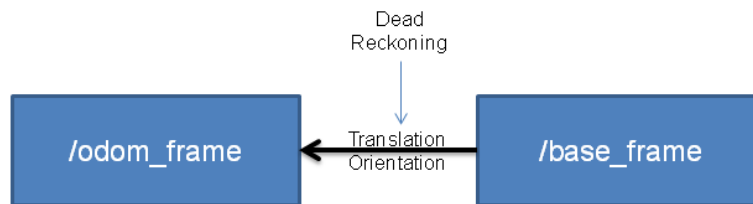
Local costmap:



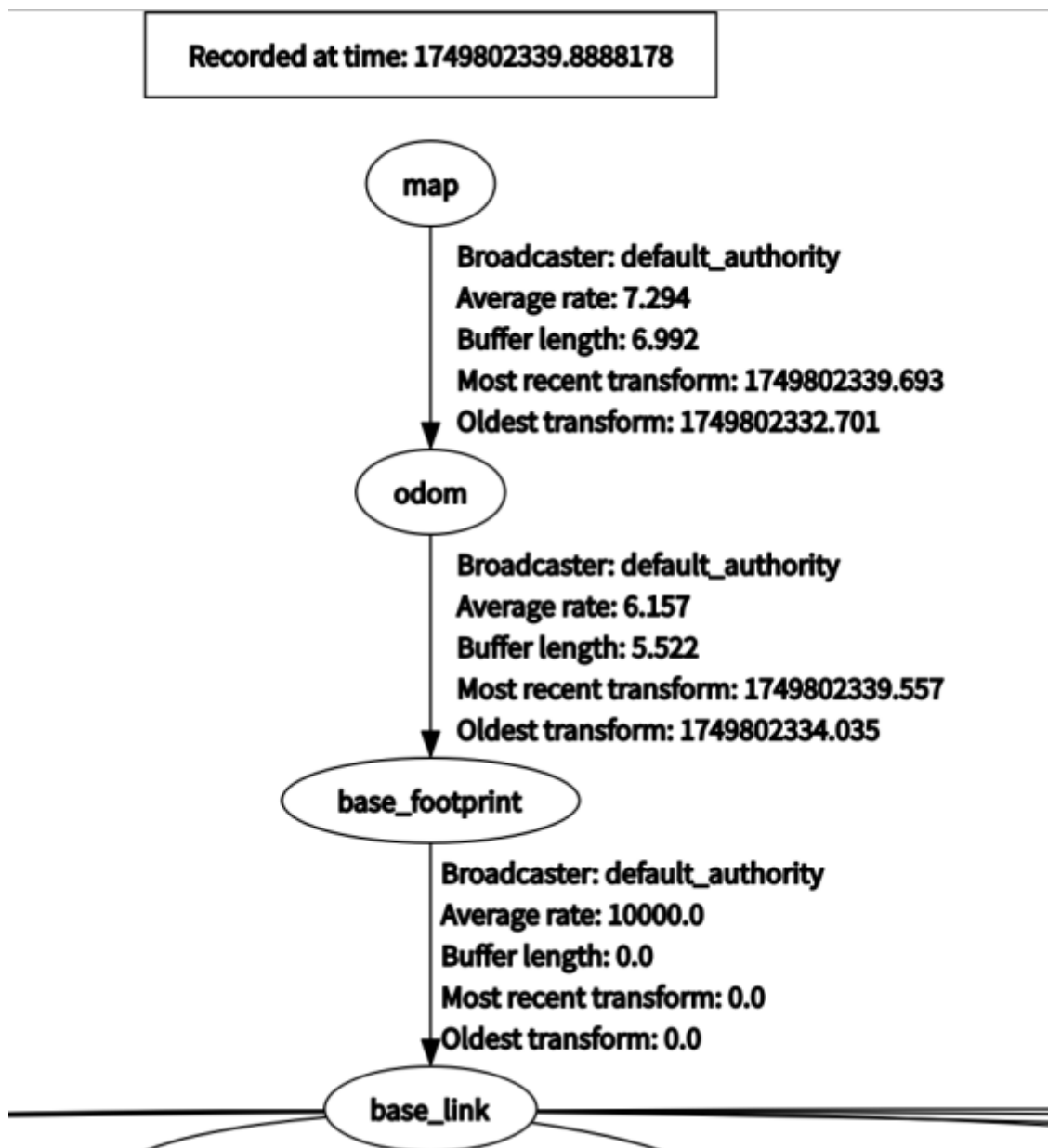
5.2.2 AMCL Localization Algorithm

AMCL (Adaptive Monte Carlo Localization) is a probabilistic localization system for 2D mobile robots. It implements an adaptive (or KLD-sampling) Monte Carlo localization method that can estimate the robot's position using a particle filter based on the existing map.

As shown in the figure below, if the odometry had no error, in the ideal case we could directly use the odometry information (upper figure) to compute the robot's (`base_frame`) position relative to the odometry frame. In reality, however, the odometry has drift and non-negligible accumulated error, so AMCL uses the method in the lower figure: it first roughly localizes `base_frame` using the odometry information, and then obtains `base_frame` relative to `map_frame` (the global map frame) through the measurement model, thus knowing the robot's pose in the map. (Note that although what is estimated here is the base-to-map transform, what is finally published is the map-to-odom transform, which can be understood as the drift of the odometry.)



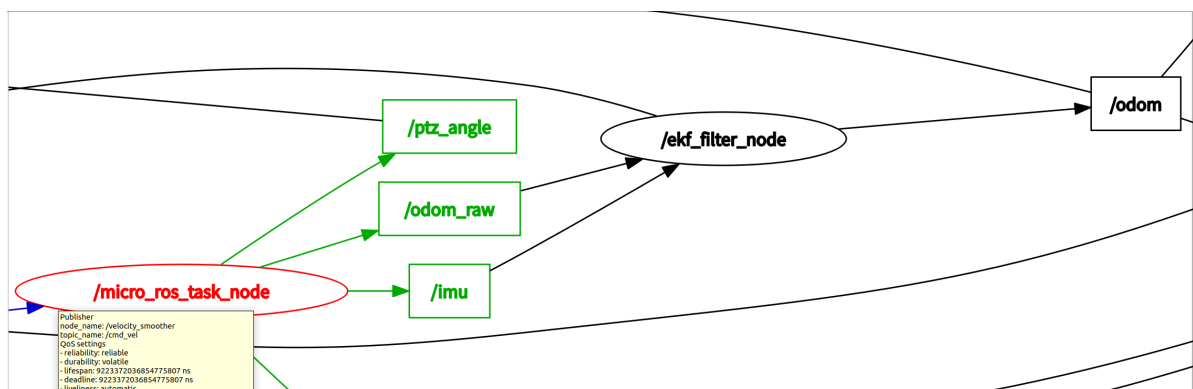
In the TF tree during robot navigation, the coordinate transform between map (map frame) and odom (odometry frame) is published by the AMCL localization algorithm, while the coordinate transform between odom (odometry frame) and base_footprint (the projection frame of the robot chassis center) is published by the **ekf_filter_node** node under the **robot_localization** package, whose role is to publish the odometry information obtained by fusing the raw IMU sensor and encoder odometry.



- In the node communication graph, we can find the communication data flow for ekf_filter_node.

In this graph, the **/micro_ros_task_node** node is the chassis node, which publishes **/imu** (raw IMU sensor data) and **/odom_raw** (raw encoder odometry data).

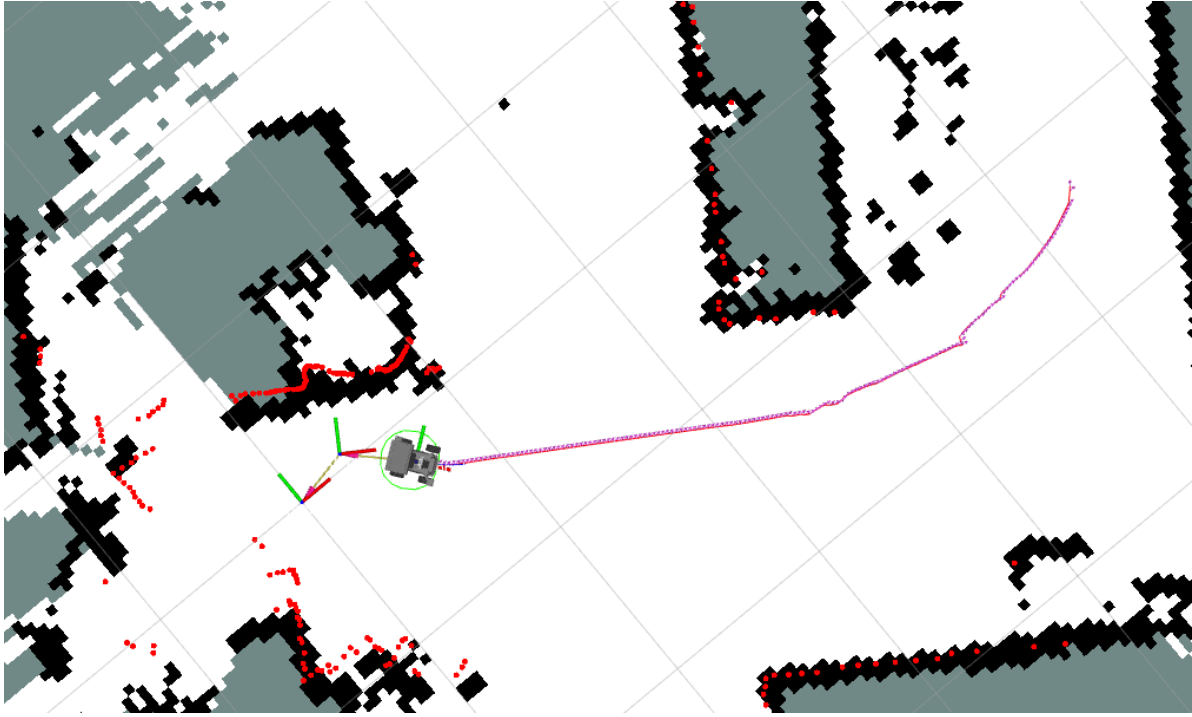
The **ekf_filter_node** node subscribes to the **/odom_raw** and **/imu** topics, fuses the multi-sensor data, and publishes the **/odom** topic.



5.3 Path Planning

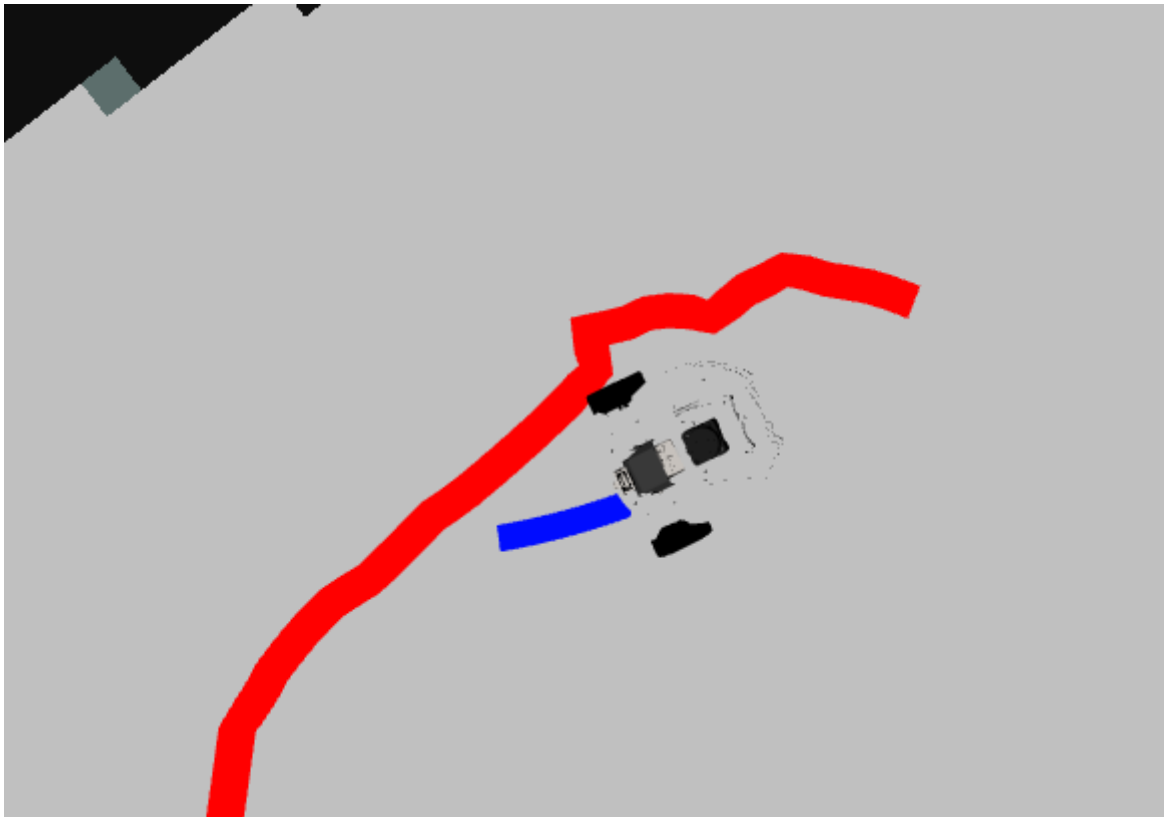
5.3.1 Global Path Planning

- The role of global path planning is to compute an optimal or feasible global path from the start point to the goal point in a known environment map, without considering real-time obstacles and aiming at global optimality. As shown in the figure below, the path connected by arrows is the global path.



5.3.2 Local Path Planning

- The role of local path planning is to dynamically adjust the path during robot motion based on real-time lidar sensor data to avoid obstacles (obstacles are displayed in the costmap). As shown in the figure below, the blue path is the local path.



6. Navigation Parameter Configuration

- Configuration file path:

```
$HOME/microros_ws/src/microros_v2_bringup/config/nav2_params.yaml
```

6.1 Navigation Speed Tuning

Parameter Tuning Guide

- If you need to adjust the linear speed and angular velocity during navigation, adjust `max_vel_x` and `max_vel_theta`.
- If you need to adjust the accuracy of reaching the goal point, adjust `xy_goal_tolerance` and `yaw_goal_tolerance`.
- For adjustments to other parameters, see the comments below.

6.2 Obstacle Avoidance Tuning

Parameter Tuning Guide

- If the robot navigates too close to obstacles on the map, you can enlarge the inflation area to keep the robot away from obstacles by increasing `inflation_radius` (inflation radius) and `cost_scaling_factor` (scaling factor — the larger the value, the less likely the robot is to approach obstacles) in `local_costmap` and `global_costmap`.
- For adjustments to other parameters, see the comments below.

```
# AMCL (Adaptive Monte Carlo Localization) configuration
amcl:
  ros__parameters:
    # whether to use simulated time; False means use real time
```

```
use_sim_time: False
# Noise parameter of the odometry model when the robot rotates
alpha1: 0.2
# Noise parameter of the odometry model for translation caused by rotation
alpha2: 0.2
# Noise parameter of the odometry model when the robot translates
alpha3: 0.2
# Noise parameter of the odometry model for rotation caused by translation
alpha4: 0.2
# Noise parameter of the odometry model when the robot rotates (another
aspect)
alpha5: 0.2
# Name of the robot base frame
base_frame_id: "base_footprint"
# Distance threshold for beam skipping, used for laser scan data processing
beam_skip_distance: 0.5
# Error threshold for beam skipping, used to decide whether to skip certain
beams
beam_skip_error_threshold: 0.9
# Threshold for beam skipping, used to decide whether to skip certain beams
beam_skip_threshold: 0.3
# Whether to enable beam skipping
do_beamskip: false
# Name of the global frame
global_frame_id: "map"
# Decay coefficient for short laser reflections
lambda_short: 0.1
# Maximum likelihood distance of the laser scan
laser_likelihood_max_dist: 2.0
# Maximum range of the laser scan
laser_max_range: 100.0
# Minimum range of the laser scan
laser_min_range: -1.0
# Laser model type
laser_model_type: "likelihood_field"
# Maximum number of laser beams used per update
max_beams: 60
# Maximum number of particles allowed in the particle filter
max_particles: 2000
# Minimum number of particles allowed in the particle filter
min_particles: 500
# Name of the odometry frame
odom_frame_id: "odom"
# Error threshold of the particle filter
pf_err: 0.05
# Confidence threshold of the particle filter
pf_z: 0.99
# Fast recovery factor for the particle filter recovery mechanism
recovery_alpha_fast: 0.0
# Slow recovery factor for the particle filter recovery mechanism
recovery_alpha_slow: 0.0
# Resampling interval
resample_interval: 1
# Robot motion model type
robot_model_type: "nav2_amcl::DifferentialMotionModel"
# Rate at which the robot pose is saved
save_pose_rate: 0.5
# Standard deviation of laser hits
```

```

sigma_hit: 0.2
# Whether to broadcast TF transforms
tf_broadcast: true
# Publish rate of TF transforms
tf_publish_rate: 10.0
# Tolerance time for transforms
transform_tolerance: 0.2
# Minimum threshold for angular updates
update_min_a: 0.2
# Minimum threshold for distance updates
update_min_d: 0.25
# Weight of laser hits
z_hit: 0.5
# Weight of the maximum laser range
z_max: 0.05
# Weight of random laser measurements
z_rand: 0.5
# Weight of short laser reflections
z_short: 0.05
# Topic name for laser scan data
scan_topic: scan

# AMCL map client configuration
amcl_map_client:
  ros__parameters:
    # Whether to use simulated time; False means use real time
    use_sim_time: False

# AMCL RCLCPP node configuration
amcl_rclcpp_node:
  ros__parameters:
    # Whether to use simulated time; False means use real time
    use_sim_time: False

# BT navigator configuration
bt_navigator:
  ros__parameters:
    # Whether to use simulated time; False means use real time
    use_sim_time: False
    # Name of the global frame
    global_frame: map
    # Name of the robot base frame
    robot_base_frame: base_link
    # Topic name for odometry data
    odom_topic: /odom
    # Default behavior tree XML file name
    default_bt_xml_filename: "navigate_w_replanning_and_recovery.xml"
    # Behavior tree loop duration
    bt_loop_duration: 10
    # Default server timeout
    default_server_timeout: 20
    # Whether to enable Groot monitoring
    enable_groot_monitoring: True
    # Groot ZMQ publisher port
    groot_zmq_publisher_port: 1666
    # Groot ZMQ server port
    groot_zmq_server_port: 1667
    # List of behavior tree plugin library names

```

```

plugin_lib_names:
- nav2_compute_path_to_pose_action_bt_node
- nav2_compute_path_through_poses_action_bt_node
- nav2_follow_path_action_bt_node
- nav2_back_up_action_bt_node
- nav2_spin_action_bt_node
- nav2_wait_action_bt_node
- nav2_clear_costmap_service_bt_node
- nav2_is_stuck_condition_bt_node
- nav2_goal_reached_condition_bt_node
- nav2_goal_updated_condition_bt_node
- nav2_initial_pose_received_condition_bt_node
- nav2_reinitialize_global_localization_service_bt_node
- nav2_rate_controller_bt_node
- nav2_distance_controller_bt_node
- nav2_speed_controller_bt_node
- nav2_truncate_path_action_bt_node
- nav2_goal_updater_node_bt_node
- nav2_recovery_node_bt_node
- nav2_pipeline_sequence_bt_node
- nav2_round_robin_node_bt_node
- nav2_transform_available_condition_bt_node
- nav2_time_expired_condition_bt_node
- nav2_distance_traveled_condition_bt_node
- nav2_single_trigger_bt_node
- nav2_is_battery_low_condition_bt_node
- nav2_navigate_through_poses_action_bt_node
- nav2_navigate_to_pose_action_bt_node
- nav2_remove_passed_goals_action_bt_node
- nav2_planner_selector_bt_node
- nav2_controller_selector_bt_node
- nav2_goal_checker_selector_bt_node

```

BT navigator RCLCPP node configuration

```
bt_navigator_rclcpp_node:
```

```
  ros__parameters:
```

```
    # Whether to use simulated time; False means use real time
    use_sim_time: False

```

Controller server configuration

```
controller_server:
```

```
  ros__parameters:
```

```
    # Whether to use simulated time; False means use real time
    use_sim_time: False
    # Controller operating frequency
    controller_frequency: 5.0
    # Minimum velocity threshold in the x direction
    min_x_velocity_threshold: 0.001
    # Minimum velocity threshold in the y direction
    min_y_velocity_threshold: 0.5
    # Minimum velocity threshold in the rotation direction
    min_theta_velocity_threshold: 0.001
    # Maximum allowed failure tolerance
    failure_tolerance: 0.3
    # Progress checker plugin name
    progress_checker_plugin: "progress_checker"
    # List of goal checker plugin names
    goal_checker_plugins: ["general_goal_checker"]

```

```

# List of controller plugin names
controller_plugins: ["FollowPath"]

# Progress checker parameters
progress_checker:
  # Progress checker plugin type
  plugin: "nav2_controller::SimpleProgressChecker"
  # Minimum radius the robot must move
  required_movement_radius: 0.5
  # Maximum allowed movement time
  movement_time_allowance: 10.0

general_goal_checker:
  # Whether the goal checker is stateful
  stateful: True
  # Goal checker plugin type
  plugin: "nav2_controller::SimpleGoalChecker"
  # Goal tolerance in the X and Y directions
  xy_goal_tolerance: 0.15
  # Goal tolerance in the rotation direction
  yaw_goal_tolerance: 0.15

# DWB local planner parameters
FollowPath:
  # Local planner plugin type
  plugin: "dwb_core::DWBLocalPlanner"
  # Whether to debug trajectory details
  debug_trajectory_details: True
  # Minimum velocity in the X direction
  min_vel_x: 0.0
  # Minimum velocity in the Y direction
  min_vel_y: 0.0
  # Maximum velocity in the X direction
  max_vel_x: 0.35
  # Maximum velocity in the Y direction
  max_vel_y: 0.0
  # Maximum velocity in the rotation direction
  max_vel_theta: 1.0
  # Minimum velocity in the XY plane
  min_speed_xy: 0.0
  # Maximum velocity in the XY plane
  max_speed_xy: 0.22
  # Minimum velocity in the rotation direction
  min_speed_theta: 0.0
  # Acceleration limit in the X direction
  acc_lim_x: 2.5
  # Acceleration limit in the Y direction
  acc_lim_y: 0.0
  # Acceleration limit in the rotation direction
  acc_lim_theta: 3.2
  # Deceleration limit in the X direction
  decel_lim_x: -2.5
  # Deceleration limit in the Y direction
  decel_lim_y: 0.0
  # Deceleration limit in the rotation direction
  decel_lim_theta: -3.2
  # Number of velocity samples in the X direction
  vx_samples: 20

```

```

# Number of velocity samples in the Y direction
vy_samples: 0
# Number of velocity samples in the rotation direction
vtheta_samples: 40
# Simulation time
sim_time: 1.5
# Linear granularity
linear_granularity: 0.05
# Angular granularity
angular_granularity: 0.025
# Tolerance time for transforms
transform_tolerance: 0.2
# Goal tolerance in the X and Y directions
xy_goal_tolerance: 0.05
# Velocity threshold for stopping translation
trans_stopped_velocity: 0.25
# Whether to short-circuit trajectory evaluation
short_circuit_trajectory_evaluation: True
# Whether the planner is stateful
stateful: True
# List of critics
critics: ["RotateToGoal", "Oscillation", "BaseObstacle", "GoalAlign",
"PathAlign", "PathDist", "GoalDist"]
# weight of the base obstacle critic
BaseObstacle.scale: 0.02
# Weight of the path alignment critic
PathAlign.scale: 32.0
# Forward point distance of the path alignment critic
PathAlign.forward_point_distance: 0.1
# Weight of the goal alignment critic
GoalAlign.scale: 24.0
# Forward point distance of the goal alignment critic
GoalAlign.forward_point_distance: 0.1
# Weight of the path distance critic
PathDist.scale: 32.0
# Weight of the goal distance critic
GoalDist.scale: 24.0
# Weight of the rotate-to-goal critic
RotateToGoal.scale: 32.0
# Slowing factor of the rotate-to-goal critic
RotateToGoal.slowing_factor: 5.0
# Lookahead time of the rotate-to-goal critic
RotateToGoal.lookahead_time: -1.0

# Controller server RCLCPP node configuration
controller_server_rclcpp_node:
  ros__parameters:
    # Whether to use simulated time; False means use real time
    use_sim_time: False

# Local costmap configuration
local_costmap:
  local_costmap:
    ros__parameters:
      # Costmap update frequency
      update_frequency: 5.0
      # Costmap publish frequency
      publish_frequency: 2.0

```

```
# Name of the global frame
global_frame: odom
# Name of the robot base frame
robot_base_frame: base_link
# Whether to use simulated time; False means use real time
use_sim_time: False
# Whether to use a rolling window
rolling_window: true
# Costmap width
width: 3
# Costmap height
height: 3
# Costmap resolution
resolution: 0.05
# Robot radius
robot_radius: 0.15
# List of costmap plugins
plugins: ["obstacle_layer", "voxel_layer", "inflation_layer"]
inflation_layer:
  # Inflation layer plugin type
  plugin: "nav2_costmap_2d::InflationLayer"
  # Inflation radius
  inflation_radius: 0.3
  # Cost scaling factor
  cost_scaling_factor: 3.0
obstacle_layer:
  # Obstacle layer plugin type
  plugin: "nav2_costmap_2d::ObstacleLayer"
  # Whether the obstacle layer is enabled
  enabled: True
  # Observation source name
  observation_sources: scan
  scan:
    # Topic name for laser scan data
    topic: /scan
    # Maximum obstacle height
    max_obstacle_height: 2.0
    # Whether to clear obstacles
    clearing: True
    # Whether to mark obstacles
    marking: True
    # Data type
    data_type: "LaserScan"
voxel_layer:
  # Voxel layer plugin type
  plugin: "nav2_costmap_2d::VoxelLayer"
  # Whether the voxel layer is enabled
  enabled: True
  # Whether to publish the voxel map
  publish_voxel_map: True
  # Origin in the Z direction
  origin_z: 0.0
  # Resolution in the Z direction
  z_resolution: 0.05
  # Number of voxels in the Z direction
  z_voxels: 16
  # Maximum obstacle height
  max_obstacle_height: 2.0
```

```

# Mark threshold
mark_threshold: 0
# Observation source name
observation_sources: scan
scan:
  # Topic name for laser scan data
  topic: /scan
  # Maximum obstacle height
  max_obstacle_height: 2.0
  # Whether to clear obstacles
  clearing: True
  # Whether to mark obstacles
  marking: True
  # Data type
  data_type: "LaserScan"
  # Maximum range for ray tracing
  raytrace_max_range: 3.0
  # Minimum range for ray tracing
  raytrace_min_range: 0.0
  # Maximum obstacle detection range
  obstacle_max_range: 2.5
  # Minimum obstacle detection range
  obstacle_min_range: 0.0
static_layer:
  # Whether to subscribe to the transient local map
  map_subscribe_transient_local: True
  # Whether to always send the full costmap
  always_send_full_costmap: True
local_costmap_client:
  ros__parameters:
    # Whether to use simulated time; False means use real time
    use_sim_time: False
local_costmap_rclcpp_node:
  ros__parameters:
    # Whether to use simulated time; False means use real time
    use_sim_time: False

# Global costmap configuration
global_costmap:
  global_costmap:
    ros__parameters:
      # Costmap update frequency
      update_frequency: 1.0
      # Costmap publish frequency
      publish_frequency: 1.0
      # Name of the global frame
      global_frame: map
      # Name of the robot base frame
      robot_base_frame: base_link
      # Whether to use simulated time; True means use simulated time
      use_sim_time: True
      # Robot radius
      robot_radius: 0.2
      # Costmap resolution
      resolution: 0.05
      # Whether to track unknown space
      track_unknown_space: false
      # List of costmap plugins

```

```
plugins: ["static_layer", "obstacle_layer", "voxel_layer",
"inflation_layer"]
obstacle_layer:
  # Obstacle layer plugin type
  plugin: "nav2_costmap_2d:ObstacleLayer"
  # Whether the obstacle layer is enabled
  enabled: True
  # Observation source name
  observation_sources: scan
  scan:
    # Topic name for laser scan data
    topic: /scan
    # Maximum obstacle height
    max_obstacle_height: 2.0
    # Whether to clear obstacles
    clearing: True
    # Whether to mark obstacles
    marking: True
    # Data type
    data_type: "LaserScan"
    # Maximum range for ray tracing
    raytrace_max_range: 3.0
    # Minimum range for ray tracing
    raytrace_min_range: 0.0
    # Maximum obstacle detection range
    obstacle_max_range: 2.5
    # Minimum obstacle detection range
    obstacle_min_range: 0.0
voxel_layer:
  # Voxel layer plugin type
  plugin: "nav2_costmap_2d:VoxelLayer"
  # Whether the voxel layer is enabled
  enabled: True
  # Whether to publish the voxel map
  publish_voxel_map: True
  # Origin in the Z direction
  origin_z: 0.0
  # Resolution in the Z direction
  z_resolution: 0.05
  # Number of voxels in the Z direction
  z_voxels: 16
  # Maximum obstacle height
  max_obstacle_height: 2.0
  # Mark threshold
  mark_threshold: 0
  # Observation source name
  observation_sources: scan
  scan:
    # Topic name for laser scan data
    topic: /scan
    # Maximum obstacle height
    max_obstacle_height: 2.0
    # Whether to clear obstacles
    clearing: True
    # Whether to mark obstacles
    marking: True
    # Data type
    data_type: "LaserScan"
```

```

    # Maximum range for ray tracing
    raytrace_max_range: 3.0
    # Minimum range for ray tracing
    raytrace_min_range: 0.0
    # Maximum obstacle detection range
    obstacle_max_range: 2.5
    # Minimum obstacle detection range
    obstacle_min_range: 0.0
static_layer:
    # Static layer plugin type
    plugin: "nav2_costmap_2d::StaticLayer"
    # Whether to subscribe to the transient local map
    map_subscribe_transient_local: True
inflation_layer:
    # Inflation layer plugin type
    plugin: "nav2_costmap_2d::InflationLayer"
    # Cost scaling factor
    cost_scaling_factor: 3.0
    # Inflation radius
    inflation_radius: 0.3
    # Whether to always send the full costmap
    always_send_full_costmap: True
global_costmap_client:
    ros__parameters:
        # Whether to use simulated time; False means use real time
        use_sim_time: False
global_costmap_rclcpp_node:
    ros__parameters:
        # Whether to use simulated time; False means use real time
        use_sim_time: False

# Map server configuration
map_server:
    ros__parameters:
        # Whether to use simulated time; False means use real time
        use_sim_time: False
        # Name of the map YAML file
        yaml_filename: "map.yaml"

# Map saver configuration
map_saver:
    ros__parameters:
        # Whether to use simulated time; False means use real time
        use_sim_time: False
        # Timeout for saving the map
        save_map_timeout: 5.0
        # Default free threshold
        free_thresh_default: 0.25
        # Default occupied threshold
        occupied_thresh_default: 0.65
        # Whether to subscribe to the transient local map
        map_subscribe_transient_local: True

# Planner server configuration
planner_server:
    ros__parameters:
        # Expected planner frequency
        expected_planner_frequency: 5.0

```

```

# Whether to use simulated time; False means use real time
use_sim_time: False
# List of planner plugin names
planner_plugins: ["GridBased"]
GridBased:
  # Grid-based planner plugin type
  plugin: "nav2_navfn_planner/NavfnPlanner"
  # Planning tolerance
  tolerance: 0.5
  # Whether to use the A* algorithm
  use_astar: false
  # Whether to allow unknown space
  allow_unknown: true

# Planner server RCLCPP node configuration
planner_server_rclcpp_node:
  ros__parameters:
    # Whether to use simulated time; False means use real time
    use_sim_time: False

# Recoveries server configuration
recoveries_server:
  ros__parameters:
    # Topic name for the costmap
    costmap_topic: local_costmap/costmap_raw
    # Topic name for the robot footprint
    footprint_topic: local_costmap/published_footprint
    # Cycle frequency
    cycle_frequency: 5.0
    # List of recovery plugin names
    recovery_plugins: ["spin", "backup", "wait"]
    spin:
      # Spin recovery plugin type
      plugin: "nav2_recoveries/Spin"
    backup:
      # Back-up recovery plugin type
      plugin: "nav2_recoveries/Backup"
    wait:
      # Wait recovery plugin type
      plugin: "nav2_recoveries/Wait"
    # Name of the global frame
    global_frame: odom
    # Name of the robot base frame
    robot_base_frame: base_link
    # Transform timeout
    transform_timeout: 0.1
    # Whether to use simulated time; False means use real time
    use_sim_time: False
    # Look-ahead simulation time
    simulate_ahead_time: 2.0
    # Maximum rotational velocity
    max_rotational_vel: 1.0
    # Minimum rotational velocity
    min_rotational_vel: 0.4
    # Rotational acceleration limit
    rotational_acc_lim: 3.2

# Robot state publisher configuration

```

```

robot_state_publisher:
  ros__parameters:
    # Whether to use simulated time; False means use real time
    use_sim_time: False

# Waypoint follower configuration
waypoint_follower:
  ros__parameters:
    # Loop rate
    loop_rate: 2000
    # Whether to stop on failure
    stop_on_failure: false
    # Waypoint task executor plugin name
    waypoint_task_executor_plugin: "wait_at_waypoint"
    wait_at_waypoint:
      # Waypoint wait plugin type
      plugin: "nav2_waypoint_follower::waitAtWaypoint"
      # Whether the plugin is enabled
      enabled: True
      # Waypoint pause duration
      waypoint_pause_duration: 200

```

7. Source Code Analysis

- Launch file path

```
$HOME/microros_ws/src/microros_v2_bringup/launch/nav2.launch.py
```

(Add program explanation here)

```

def generate_launch_description():
    map_folder_dir = os.path.join(os.path.expanduser('~'), 'map')
    bringup_dir=os.path.join(get_package_share_directory('microros_v2_bringup'))
    declared_arguments = []
    declared_arguments.append(
        DeclareLaunchArgument(
            "map",
            default_value="map.yaml",
            description="map name",
        )
    )
    declared_arguments.append(
        DeclareLaunchArgument(
            "rviz",
            default_value='True',
            description="if rviz is used, set to True. If False, rviz will not
be launched.",
        )
    )

    declared_arguments.append(
        DeclareLaunchArgument(

```

```

        "start_carbase",
        default_value='True',
        description="Whether to launch Carbase is determined by the
navigation node, for integration with other launchers",
    )
)

declared_arguments.append(
    DeclareLaunchArgument(
        "log_level",
        default_value='info',
        description="ROS log level: debug, info, warn, error, fatal",
    )
)

#cartographer state file
declared_arguments.append(
    DeclareLaunchArgument(
        "state_filename",
        default_value="map.pbstream",
        description="=====",
    )
)

declared_arguments.append(
    DeclareLaunchArgument(
        "localization_mode",
        default_value="amcl",
        description="localization_mode ,amcl or cartographer",
    )
)

#namespace
declared_arguments.append(
    DeclareLaunchArgument(
        "namespace",
        default_value="",
        description="Add namespace",
    )
)

#slam mode
declared_arguments.append(
    DeclareLaunchArgument(
        "slam",
        default_value="False",
        description="Start SLAM simultaneously with navigation",
    )
)

declared_arguments.append(
    DeclareLaunchArgument(
        "pose_map",
        default_value="map",
        description="",
    )
)

# Initialize variables / Initialize Arguments
map = LaunchConfiguration("map")
map_file = PathJoinSubstitution([map_folder_dir, map])

```

```

pose_map=PathJoinSubstitution([map_folder_dir,
LaunchConfiguration("pose_map")])
rviz = LaunchConfiguration("rviz")
slam = LaunchConfiguration('slam')
localization_mode=LaunchConfiguration("localization_mode")
state_filename_file = PathJoinSubstitution([map_folder_dir,
LaunchConfiguration("state_filename")])
namespace=LaunchConfiguration("namespace")
log_level=LaunchConfiguration('log_level')

nav_rviz_config=os.path.join(bringup_dir, 'rviz', 'nav2.rviz')
params_file=os.path.join(bringup_dir, "config", "nav2_params.yaml")

#navigation2 startup file
nav2_launch_file = os.path.join(get_package_share_directory("nav2_bringup"),
                                "launch",
                                'bringup_launch.py')

#The modified startup file of navigation2 blocks the AMCL positioning
function
nav2_launch_mod = os.path.join(bringup_dir,
                                'launch',
                                'bringup_mod.launch.py'
                                )

carbase=IncludeLaunchDescription(
    PythonLaunchDescriptionSource(os.path.join(bringup_dir, 'launch', 'car_base.launch
h.py')),
    condition=IfCondition(LaunchConfiguration('start_carbase')),
    launch_arguments={ 'log_level': 'info',
                        'use_camera': 'False'
                        }.items())

#AMCL positioning method navigation
amcl_localization_nav = GroupAction(
    condition=IfCondition(PythonExpression(["'", localization_mode, "' ==
'amcl' and '", slam, "' == 'False' "]])),
    actions=[
        IncludeLaunchDescription(
            PythonLaunchDescriptionSource(nav2_launch_file),
            launch_arguments={
                "map": map_file,
                "params_file": params_file,
                "use_sim_time": "False",
                "namespace": namespace,
                "log_level": log_level,
            }.items(),
        )
    ]
)

#Cartographer positioning navigation
cartographer_localization_nav = GroupAction(
    condition=IfCondition(PythonExpression(["'", localization_mode, "' ==
'cartographer' and '", slam, "' == 'False' "]])),
    actions=[
        #Start navigation2 navigation file (no global positioning)

```

```

PushRosNamespace(namespace=namespace),
IncludeLaunchDescription(
    PythonLaunchDescriptionSource(nav2_launch_mod),
    launch_arguments={
        "map": map_file,
        "params_file": params_file,
        "use_sim_time": "False",
        "namespace": namespace,
        "log_level": log_level,
    }.items()),
Node(
    package = 'cartographer_ros',
    executable = 'cartographer_node',
    parameters = [{'use_sim_time': False}],
    arguments = [
        '-configuration_directory',
os.path.join(get_package_share_directory('microros_v2_bringup'), 'config'),
        '-configuration_basename',
'cartographer_2d_localization.lua',
        '-load_state_filename', state_filename_file],
    remappings = [('imu', '/imu/data_raw')],
    output = 'screen'
),
Node(
    package = 'cartographer_ros',
    executable = 'cartographer_occupancy_grid_node',
    parameters = [
        {'use_sim_time': False},
        {'resolution': 0.05}],
)
]
)

slam_toolbox_localization_nav = GroupAction(
    condition=IfCondition(PythonExpression(["'", localization_mode, "' == 'slam_toolbox' and '", slam, "' == 'False' "]])),
    actions=[
        #Start navigation2 navigation file (no global positioning)
        PushRosNamespace(namespace=namespace),
        IncludeLaunchDescription(
            PythonLaunchDescriptionSource(nav2_launch_mod),
            launch_arguments={
                "map": map_file,
                "params_file": params_file,
                "use_sim_time": "False",
                "namespace": namespace,
                "log_level": log_level,
            }.items()),
        Node(
            package='slam_toolbox',
            executable='localization_slam_toolbox_node',
            name='slam_toolbox',
            parameters=[
os.path.join(bringup_dir, 'config', 'mapper_params_localization.yaml'),
                {'use_sim_time': False},
                {'map_file_name': pose_map}
            ],

```

```

        arguments=['--ros-args', '--log-level', 'warn'],
        output='screen')
    ]
)

#rviz
rviz_node = Node(
    package = 'rviz2',
    executable = 'rviz2',
    name='rviz2',
    arguments=['-d', nav_rviz_config,
               '--ros-args', '--log-level', log_level],
    output='screen',
    condition=IfCondition(PythonExpression(["'", rviz, "' == 'True' "]))
)

#Navigate while building the map
nav_and_slam = GroupAction(
    condition=IfCondition(PythonExpression(["'", localization_mode, "' ==
'amcl' and '", slam, "' == 'True' "]])),
    actions=[
        # Navigation using AMCL positioning method
        IncludeLaunchDescription(
            PythonLaunchDescriptionSource(nav2_launch_file),
            launch_arguments={
                "map": map_file,
                "use_sim_time": "False",
                "params_file": params_file,
                'slam': 'True',
                "namespace": namespace,
                "log_level": log_level,
            }.items(),
        )
    ]
)

return LaunchDescription(
    [
        *declared_arguments,
        rviz_node,
        carbase,
        amcl_localization_nav,
        cartographer_localization_nav,
        slam_toolbox_localization_nav,
        nav_and_slam
    ]
)

```

